LAB REPORT 2: PROCESS OPERATIONS & CONTROL

Course: Operating Systems Lab

Student Name: Aryan Nautiyal
Roll Number: 251210015

Date: 2026-08-05
Environment: Bash / POSIX (Linux x86_64)

Aim

To implement a C program in a POSIX/Linux environment demonstrating the fundamental process control primitives:

1. **Process Creation:** Duplicating execution context via `fork()`.
2. **Program Execution:** Overlaying process image memory via `execlp()`.
3. **Child Handling & Synchronization:** Reaping state & preventing zombies using `wait()`.
4. **Process Termination:** Contrasting standard exit routines (`exit()`) vs kernel-level raw exit (`_exit()`) along with child status code recovery.
5. **Asynchronous Event Handling:** Inter-process event signaling using `signal()` and `kill()`.

POSIX System Call Reference

System Call	Syntax / Prototype	OS Level Function & Behavior
`fork()`	`pid_t fork(void)`	Clones calling process. Returns 0 to child, child PID to parent, or −1 on error.
`execlp()`	`int execlp(const char *file, ...)`	Overlays caller process space with binary found via `$PATH`. Returns −1 only on error.
`wait()`	`pid_t wait(int *wstatus)`	Blocks parent until any child terminates; retrieves child exit status bitmask.
`exit()`	`void exit(int status)`	User-space clean shutdown: flushes stdio streams, runs `atexit` routines, calls `_exit()`.
`_exit()`	`void _exit(int status)`	Immediate kernel termination: releases memory & file handles, skips stdio cleanup.
`signal()`	`sighandler_t signal(int sig, ...)`	Registers a custom interrupt handler callback function for signal number `sig`.
`kill()`	`int kill(pid_t pid, int sig)`	Sends a software interrupt signal `sig` to target process specified by `pid`.

C Source Code (`process_operations.c`)

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <signal.h>
#include <errno.h>

void handle_sigusr1(int sig) {
    if (sig == SIGUSR1) {
        const char msg[] = "\n[SIGNAL HANDLER] Captured SIGUSR1! Event processed
asynchronously.\n";
        write(STDOUT_FILENO, msg, sizeof(msg) - 1);
    }
}

int main(void) {
    pid_t pid;
    int status;
    if (signal(SIGUSR1, handle_sigusr1) == SIG_ERR) {
        perror("Failed to register SIGUSR1 signal handler");
        exit(EXIT_FAILURE);
```

```
    }
    printf("[PARENT] Main Process active. PID: %d, Parent PID: %d\n", getpid(), getppid());
    pid = fork();
    if (pid < 0) {
        perror("fork() process creation failed");
        exit(EXIT_FAILURE);
    }
    else if (pid == 0) {
        printf("  [CHILD] Created successfully. PID: %d, Parent PID: %d\n",
               getpid(), getppid());
        printf("  [CHILD] Sending SIGUSR1 event signal to Parent (PID: %d)...\n", getppid());
        if (kill(getppid(), SIGUSR1) == -1) {
            perror("  [CHILD] Failed to send SIGUSR1 signal");
        }
        printf("  [CHILD] Binary Command: 'ls -l -h'\n\n");
        execlp("ls", "ls", "-l", "-h", NULL);
        _exit(127);
    }
    else {
        printf("[PARENT] Spawned child process with PID: %d\n", pid);
        printf("[PARENT] Entering blocked state waiting for child process termination...\n");
        pid_t terminated_pid = wait(&status);
        if (terminated_pid == -1) {
            perror("[PARENT] wait() system call failed");
            exit(EXIT_FAILURE);
        }
        printf("\n[PARENT] Successfully harvested Child Process (PID: %d).\n",
terminated_pid);
        if (WIFEXITED(status)) {
            printf("[PARENT] Child Exit Status: Normal exit with code [%d]. Now parent can
terminate\n",
                   WEXITSTATUS(status));
        } else if (WIFSIGNALED(status)) {
            printf("[PARENT] Child Exit Status: Abnormally terminated by Signal [%d].\n",
                   WTERMSIG(status));
        }
        exit(EXIT_SUCCESS);
    }
    return 0;
}
```

Output

```
$ gcc -Wall -Wextra process_operations.c -o process_operations
$ ./process_operations

[PARENT] Main Process active. PID: 41205, Parent PID: 18402
[PARENT] Spawned child process with PID: 41206
[PARENT] Entering blocked state waiting for child process termination...
  [CHILD] Created successfully. PID: 41206, Parent PID: 41205
  [CHILD] Sending SIGUSR1 event signal to Parent (PID: 41205)...

[SIGNAL HANDLER] Captured SIGUSR1! Event processed asynchronously.
  [CHILD] Binary Command: 'ls -l -h'

total 32K
-rwxr-xr-x 1 aryan aryan  16K Aug  4 20:45 process_operations
-rw-r--r-- 1 aryan aryan 2.8K Aug  4 20:44 process_operations.c

[PARENT] Successfully harvested Child Process (PID: 41206).
[PARENT] Child Exit Status: Normal exit with code [0]. Now parent can terminate
```